

Arquetipos Maven - Tienda Retail Lumina

1. Descripcion General

Los microservicios del proyecto Lumina se construyeron siguiendo un **arquetipo Maven estandarizado** basado en Spring Boot 3.2.5 con Java 17. Este arquetipo define la estructura base, dependencias y configuraciones comunes que todos los microservicios comparten.

2. Arquetipo Base: Microservicio Spring Boot

2.1 Configuracion Maven (pom.xml)

Todos los microservicios heredan del parent de Spring Boot:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
</parent>

<groupId>com.lumina</groupId>
<artifactId>ms-{nombre}</artifactId>
<version>1.0.0</version>

<properties>
  <java.version>17</java.version>
</properties>
```

2.2 Dependencias Comunes

Dependencia	ArtifactId	Proposito
Spring Web	spring-boot-starter-web	API REST con Tomcat embebido
Spring Data JPA	spring-boot-starter-data-jpa	ORM y acceso a datos
Spring Validation	spring-boot-starter-validation	Validacion de entrada
MySQL Connector	mysql-connector-j	Driver de base de datos
Lombok	lombok	Reduccion de boilerplate
Spring Test	spring-boot-starter-test	Testing unitario e integracion

2.3 Plugin de Build

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
```

```

                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </exclude>
        </excludes>
    </configuration>
</plugin>
</plugins>
</build>

```

3. Estructura de Directorios del Arquetipo

```

ms-{nombre}/
├─ pom.xml
├─ Dockerfile
├─ src/
│   └─ main/
│       ├── java/com/lumina/{nombre}/
│       │   ├── Ms{Nombre}Application.java    # @SpringBootApplication
│       │   ├── controller/
│       │   │   └── {Nombre}Controller.java    # @RestController
│       │   ├── service/
│       │   │   └── {Nombre}Service.java        # @Service
│       │   ├── repository/
│       │   │   └── {Nombre}Repository.java      # JpaRepository
│       │   ├── entity/
│       │   │   └── {Nombre}.java                # @Entity
│       │   ├── dto/
│       │   │   └── {Nombre}Dto.java             # Data Transfer Objects
│       └── resources/
│           └── application.yml                  # Configuración Spring
└─ test/
    └─ java/com/lumina/{nombre}/
        └── Ms{Nombre}ApplicationTests.java

```

4. Configuración Base (application.yml)

```

spring:
  application:
    name: ms-{nombre}
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:{puerto}/db_{nombre}}
    username: ${SPRING_DATASOURCE_USERNAME:lumina_user}
    password: ${SPRING_DATASOURCE_PASSWORD:lumina_pass}
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
      show-sql: true

server:
  port: {puerto}

```

5. Dockerfile Estandar (Multi-Stage Build)

```
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN apk add --no-cache maven && mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE {puerto}
ENTRYPOINT ["java", "-jar", "app.jar"]
```

6. Como Generar un Nuevo Microservicio

Para crear un nuevo microservicio siguiendo este arquetipo:

Paso 1: Crear la estructura de directorios

```
mkdir -p ms-nuevo/src/main/java/com/lumina/nuevo/{controller,service,repository,entity,dto}
mkdir -p ms-nuevo/src/main/resources
mkdir -p ms-nuevo/src/test/java/com/lumina/nuevo
```

Paso 2: Crear el pom.xml

Copiar el `pom.xml` de cualquier microservicio existente (ej: `ms-bodega/pom.xml`) y modificar:

- `artifactId` → `ms-nuevo`
- `name` → `ms-nuevo`
- `description` → Descripción del servicio

Paso 3: Crear la clase principal

```
package com.lumina.nuevo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MsNuevoApplication {
    public static void main(String[] args) {
        SpringApplication.run(MsNuevoApplication.class, args);
    }
}
```

Paso 4: Configurar application.yml

Copiar de un servicio existente y ajustar:

- `spring.application.name`
- `server.port` (asignar puerto unico)

- URL de datasource

Paso 5: Crear el Dockerfile

Copiar de un servicio existente y ajustar el `EXPOSE` al puerto del nuevo servicio.

Paso 6: Agregar al docker-compose.yml

```
ms-nuevo:
  build: ./ms-nuevo
  container_name: lumina-ms-nuevo
  ports:
    - "{puerto}:{puerto}"
  environment:
    - SPRING_PROFILES_ACTIVE=docker
    - SPRING_DATASOURCE_URL=jdbc:mysql://mysql-nuevo:3306/db_nuevo
    - SPRING_DATASOURCE_USERNAME=lumina_user
    - SPRING_DATASOURCE_PASSWORD=lumina_pass
  depends_on:
    mysql-nuevo:
      condition: service_healthy
  networks:
    - lumina-network
```

Paso 7: Agregar proxy en BFF Gateway

Crear un nuevo `ProxyController` en `bff-gateway` que redirija las peticiones al nuevo microservicio.

7. Microservicios Generados con este Arquetipo

Microservicio	Puerto App	Puerto BD	Entidades Principales
ms-usuarios	8081	3307	Usuario, Rol
ms-productos	8082	3308	Producto, Marca
ms-bodega	8083	3309	Inventario, AlertaStock
ms-carrito	8084	3310	Carrito, ItemCarrito
ms-delivery	8085	3311	Delivery, HistorialDelivery
ms-pagos	8086	3312	Pago, EstadoPago

8. Variante: Arquetipo BFF Gateway

El BFF Gateway usa una variante del arquetipo sin capa de persistencia:

Diferencias con el arquetipo base:

- No incluye `spring-boot-starter-data-jpa`
- No incluye `mysql-connector-j`
- Agrega `spring-boot-starter-webflux` (cliente HTTP reactivo)

- Agrega `spring-boot-starter-actuator` (health checks)
- No tiene carpetas `entity/` , `repository/`
- Tiene controladores Proxy que redirigen al microservicio correspondiente